Optimization of Deep Learning Pipelines

From Low-Level C-API Graph Construction to Distributed Data Parallelism

Author

Akash Saha

Department of Electronics and Communication Engineering
National Institute of Technology Rourkela

Supervisor

Preeti Malakar

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

Acknowledgement

I would like to express my sincere gratitude to my internship supervisor, **Dr. Preeti Malakar**, Assistant Professor, Department of Computer Science and Engineering, Indian Institute of Technology Kanpur, for her invaluable guidance, continuous encouragement, and insightful feedback throughout the course of this internship. Her expertise and thoughtful suggestions greatly contributed to shaping the direction and technical depth of this work.

Contents

Abstract

This report presents a systematic investigation into performance optimization techniques for deep learning workloads from a High Performance Computing (HPC) perspective. The study explores optimization across abstraction layers, ranging from high-level Python frameworks to low-level execution using the TensorFlow C API and distributed data-parallel training. The work is structured into three experimental phases: (1) profiling stochastic gradient descent behavior and batch-size–dependent performance in Python-based pipelines, (2) manual construction and execution of computational graphs using the TensorFlow C API to eliminate interpreter overhead, and (3) deployment of synchronous data-parallel training for a ResNet-18 architecture on the CIFAR-10 dataset. Experimental observations demonstrate that C-level graph execution achieves sub-millisecond step latency for simple models, while distributed training significantly reduces wall-clock time for deep architectures. The results highlight the complementary roles of low-level system optimization and high-level distributed abstractions in modern deep learning systems.

Chapter 1

Introduction

1.1 Motivation

The increasing depth and complexity of modern neural networks have imposed significant computational demands on training and inference pipelines. From an HPC standpoint, achieving optimal performance requires understanding and controlling execution beyond the Python layer, down to the compiled runtime and hardware parallelism. This motivates the exploration of low-level graph execution via the TensorFlow C API, as well as scalable training strategies that exploit data parallelism across multiple devices.

1.2 Problem Statement

Despite the maturity of deep learning frameworks, practical deployments frequently suffer from:

- Inefficient batch-size selection leading to underutilized hardware or unstable convergence.
- Overhead when deploying Python-trained models into C/C++ inference environments.
- Communication and synchronization bottlenecks in distributed training.

1.3 Project Scope

This work addresses these challenges through:

1. Profiling and benchmarking Python versus C++ inference pipelines.
2. Explicit graph construction and optimization using the TensorFlow C API.
3. Scalable training using synchronous data parallelism for deep convolutional models.

Chapter 2

Literature Review and Theoretical Background

2.1 Stochastic Gradient Descent and Batch Size

Stochastic Gradient Descent (SGD) updates parameters using gradients computed over mini-batches:

$$w_{t+1} = w_t - \eta \frac{1}{B} \sum_{i=1}^{B} \nabla L(x_i, w_t) \tag{2.1}$$

Smaller batch sizes introduce gradient noise that may improve generalization, while larger batch sizes increase arithmetic intensity and hardware efficiency. Optimal batch size selection is therefore a trade-off between statistical efficiency and computational throughput.

2.2 Residual Learning

Residual Networks mitigate vanishing gradients by introducing identity shortcut connections:

$$y = F(x, \{W_i\}) + x \tag{2.2}$$

This formulation allows gradients to propagate directly through the identity path, enabling the successful training of deep architectures such as ResNet-18.

2.3 Data Parallelism

In data-parallel training, identical replicas of the model are instantiated across multiple devices or workers, with each replica processing a disjoint subset of the global mini-batch. During each training iteration, gradients are computed independently on each replica and subsequently synchronized using collective communication operations, such as all-reduce. On GPU-based systems, this synchronization is typically implemented using high-performance backends like NCCL, which efficiently aggregate gradients across devices to ensure consistent parameter updates and maintain model equivalence across replicas.

Chapter 3

Profiling Analysis: Python vs. C++

3.1 Overview

The first phase of experimentation focused on benchmarking the inference latency gap between high-level Python execution and low-level C++ execution. The objective was to determine if bypassing the Python Global Interpreter Lock (GIL) via C++ execution yields significant latency improvements for standard workloads. A Convolutional Neural Network (CNN) trained on the MNIST dataset served as the benchmark model.

3.2 Implementation Details

The experimental pipeline was divided into two distinct stages: Model Training (Python) and Model Inference (C++).

3.2.1 1. Python Training Pipeline

The model was constructed using the Keras Sequential API.

Listing 3.1: Model Definition in Python (mnist_cnn.py)

```
# High-level Keras definition
model = keras.Sequential(
    [
        keras.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation="softmax"),
    ]
)
model.summary()
```

3.2.2 2. C++ Inference Pipeline

The C++ implementation utilizes the ‘tensorflow/cc‘ library to manually initialize the session.

Listing 3.2: Graph Loading and Session Initialization (mnist_mlp_c_api.cpp)

```
// Initialize the Session and Graph
Session* session;
Status status = NewSession(SessionOptions(), &session);
GraphDef graph_def;

// Load the frozen graph model
status = ReadBinaryProto(Env::Default(), model_path, &graph_def);
if (!status.ok()) {
    std::cerr << "Error loading model: " << status.ToString() <<
        std::endl;
    return -1;
}

// Add the graph to the session
status = session->Create(graph_def);
if (!status.ok()) {
    std::cerr << "Error creating session: " << status.ToString()
        << std::endl;
    return -1;
}
```

3.3 Performance Analysis

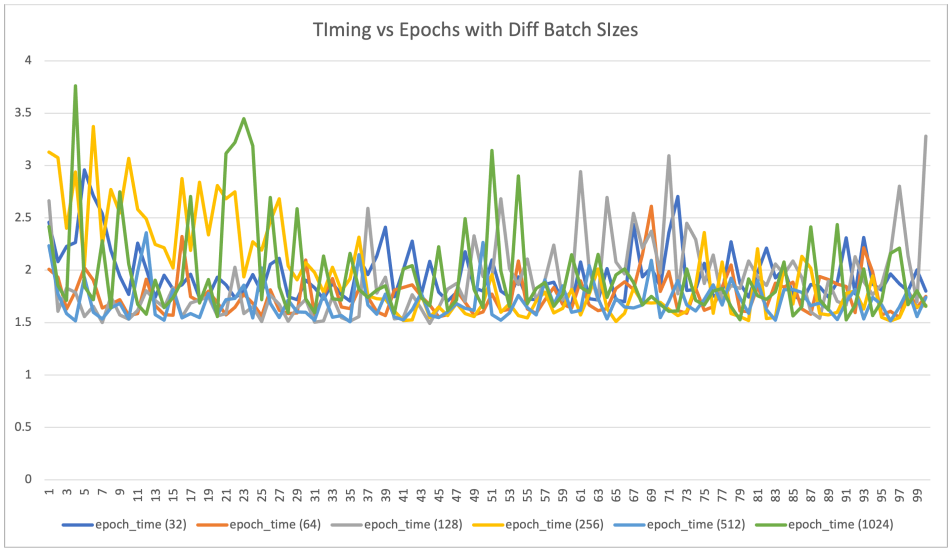

Figure 3.1: C++ Inference Timings across batch sizes.

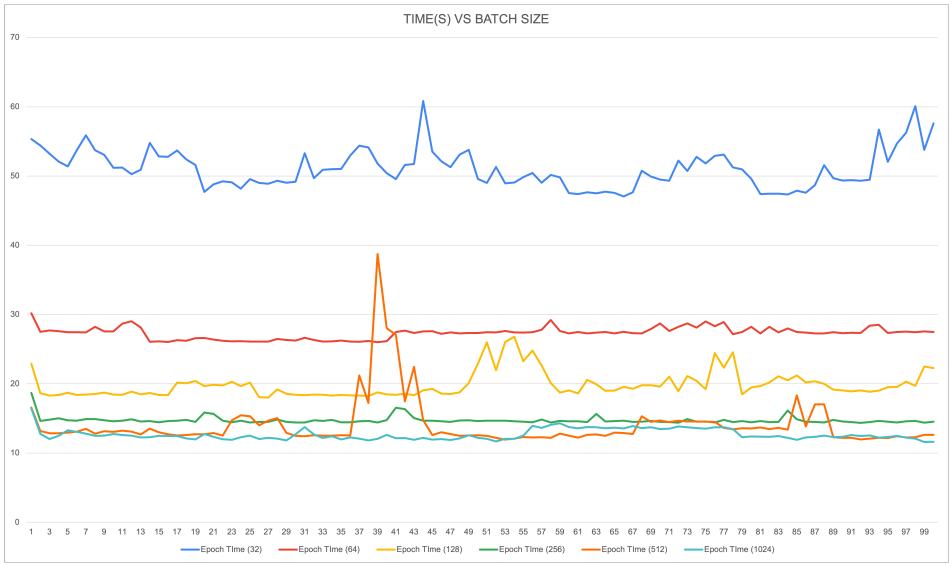

Figure 3.2: Impact of batch size on Training Time per Epoch.

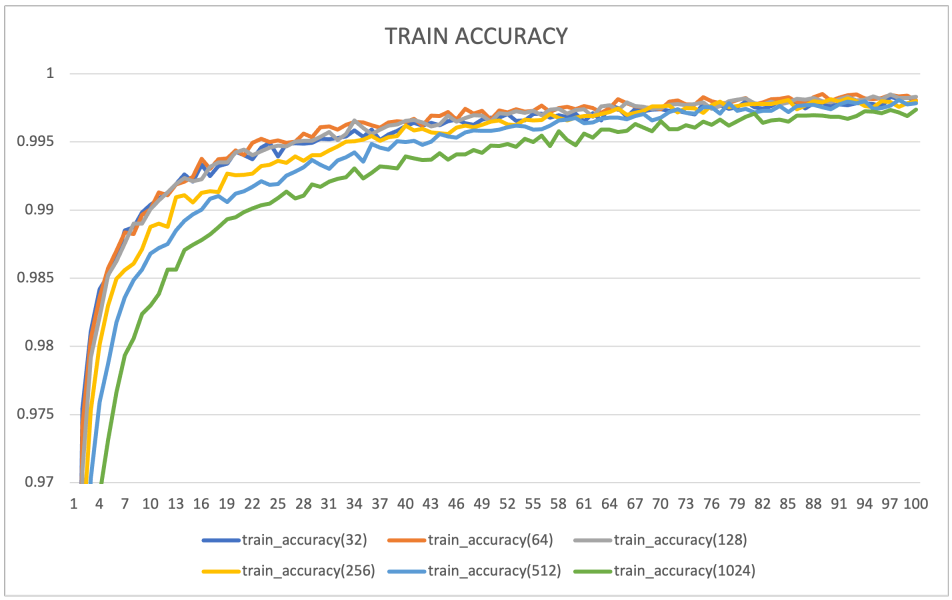

Figure 3.3: Training Accuracy convergence.

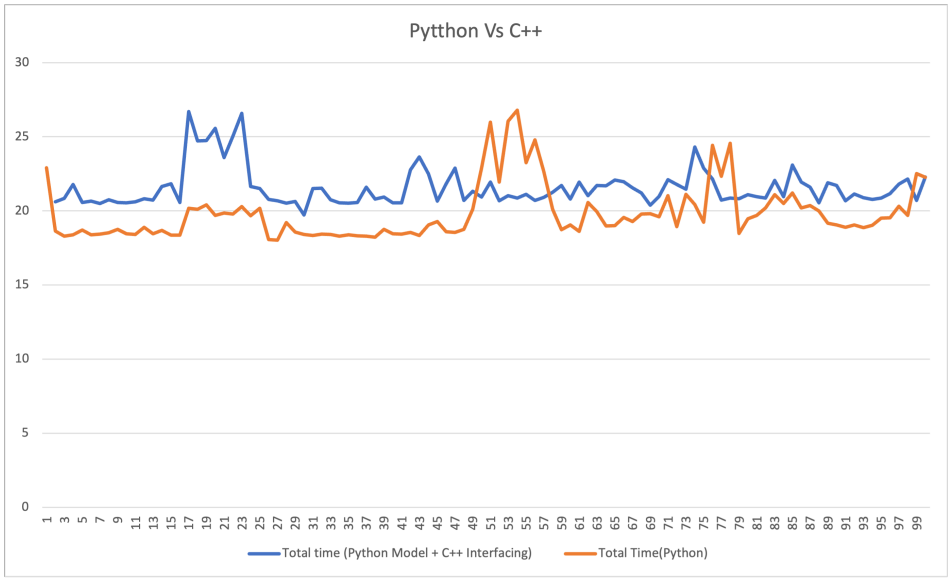

Figure 3.4: Total Execution Time Comparison: Python vs. C++.

Chapter 4

Low-Level Implementation Using TensorFlow C API

4.1 Concept

This experiment involved constructing a computational graph *from scratch* using the **TensorFlow C API** (`tensorflow/c/c_api.h`). This approach mimics how high-level frameworks like PyTorch or Keras are implemented internally.

4.2 Code Breakdown

4.2.1 1. Operation Helper Functions

Listing 4.1: Helper Function for Variable Creation (regression_c_api.c)

```c
/* Creates a trainable variable operation */

TF_Operation* Variable(TF_Graph* g, TF_Status* s, const char* name
    ) {
    TF_OperationDescription* d =
        TF_NewOperation(g, "VariableV2", name);
    TF_SetAttrType(d, "dtype", TF_FLOAT);
    TF_SetAttrShape(d, "shape", NULL, 0); // Scalar variable
    return TF_FinishOperation(d, s);

}

/* Creates a Placeholder for input data */

TF_Operation* Placeholder(TF_Graph* g, TF_Status* s, const char*
    name) {
    TF_OperationDescription* d =
        TF_NewOperation(g, "Placeholder", name);
    TF_SetAttrType(d, "dtype", TF_FLOAT);
    return TF_FinishOperation(d, s);
}
```

4.2.2 2. Manual Graph Construction

Listing 4.2: Constructing the Forward Pass (regression_c_api.c)

```
/* Compute W * x */
TF_OperationDescription* mul_d = TF_NewOperation(graph, "Mul", "
    mul");
TF_AddInput(mul_d, (TF_Output){W, 0});
TF_AddInput(mul_d, (TF_Output){x, 0});
TF_Operation* Wx = TF_FinishOperation(mul_d, status);

/* Compute (W * x) + b */
TF_OperationDescription* add_d = TF_NewOperation(graph, "Add", "
    add");
TF_AddInput(add_d, (TF_Output){Wx, 0});
TF_AddInput(add_d, (TF_Output){b, 0});
TF_Operation* y_pred = TF_FinishOperation(add_d, status);
```

4.2.3 3. Gradient Registration and Optimization

Listing 4.3: Gradient Registration and Update (regression_c_api.c)

```
/* Gradients of loss with respect to W and b */
TF_Output ys[1] = {{loss, 0}};
TF_Output xs[2] = {{W, 0}, {b, 0}};
TF_Output grads[2];

// Symbolic differentiation engine
TF_AddGradients(graph, ys, 1, xs, 2, NULL, status, grads);
Check(status);

/* Apply Gradient Descent: W = W - lr * grad */
TF_OperationDescription* applyW_d =
    TF_NewOperation(graph, "ApplyGradientDescent", "applyW");
TF_AddInput(applyW_d, (TF_Output){W, 0});
TF_AddInput(applyW_d, (TF_Output){lr, 0});
TF_AddInput(applyW_d, grads[0]);
TF_Operation* applyW = TF_FinishOperation(applyW_d, status);
```

4.3 Results: Step Latency

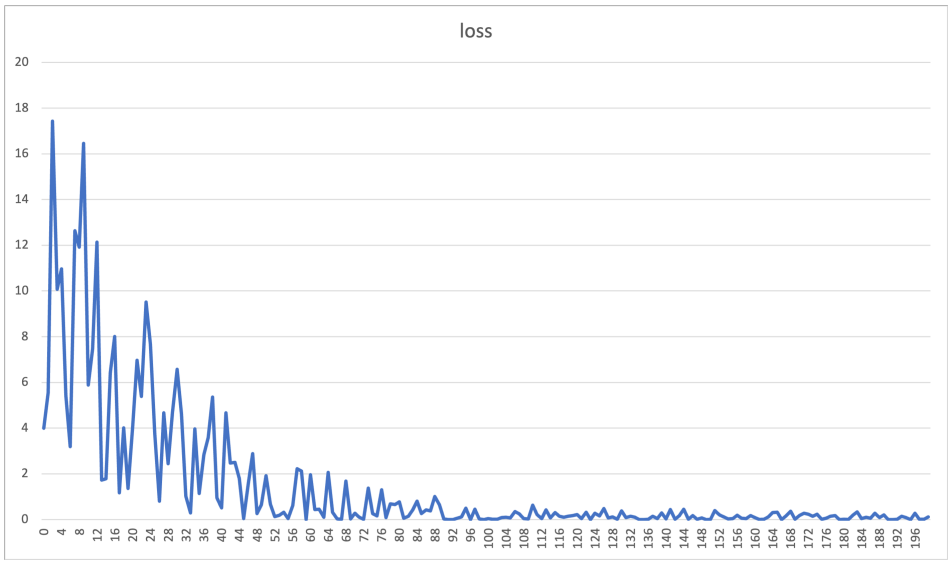

Figure 4.1: Loss convergence using the C API.

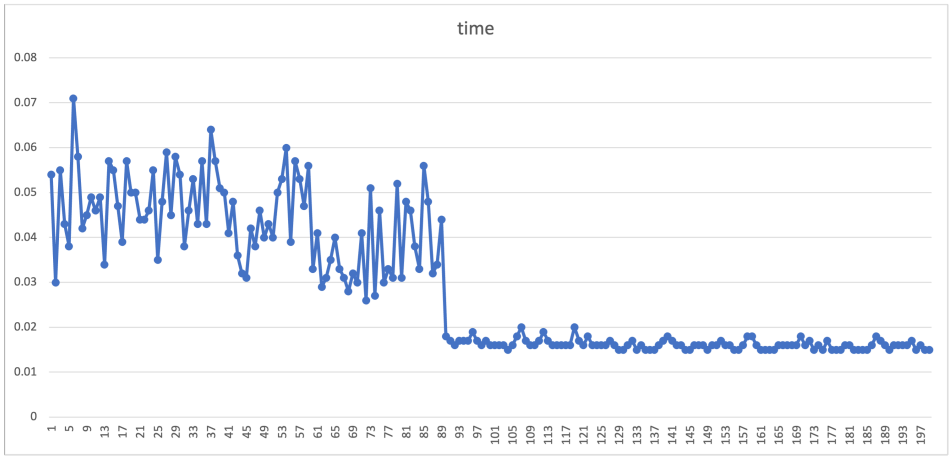

Figure 4.2: Time Per Epoch(ms).

Chapter 5

Distributed Training: ResNet-18 with Data Parallelism

5.1 Strategy Overview

The final phase focused on Data Parallelism to scale training for complex models. We implemented a ResNet-18 architecture on the CIFAR-10 dataset using MirroredStrategy.

5.2 Architecture Implementation

Listing 5.1: ResNet Block with Skip Connections (Data_Parallelization.ipynb)

```
def resnet_block(x, filters, stride=1):
    shortcut = x

    # First Convolutional Layer
    x = layers.Conv2D(filters, 3, strides=stride, padding="same",
                      use_bias=False,
                      kernel_regularizer=keras.regularizers.l2(1e
                          -4))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    # Second Convolutional Layer
    x = layers.Conv2D(filters, 3, padding="same", use_bias=False,
                      kernel_regularizer=keras.regularizers.l2(1e
                          -4))(x)
    x = layers.BatchNormalization()(x)

    # Handle dimension mismatch in shortcut path
    if stride != 1 or shortcut.shape[-1] != filters:
        shortcut = layers.Conv2D(filters, 1, strides=stride,
           use_bias=False,
                                 kernel_regularizer=keras.
                                    regularizers.l2(1e-4))(
                                    shortcut)
        shortcut = layers.BatchNormalization()(shortcut)

```

```
    # The Add() layer implements the residual connection
    x = layers.Add()([x, shortcut])
    x = layers.Activation("relu")(x)
    return x
```

5.3 Distributed Scope and Compilation

Listing 5.2: MirroredStrategy Scope (Data_Parallelization.ipynb)

```
# Initialize Strategy
strategy = tf.distribute.MirroredStrategy()
print("GPUs in sync:", strategy.num_replicas_in_sync)

# Context manager for distributed execution
with strategy.scope():
    model = build_resnet18_cifar()

    # Cosine Decay for Learning Rate
    lr_schedule = keras.optimizers.schedules.CosineDecay(
        initial_learning_rate=0.1,
        decay_steps=50000
    )
    optimizer = keras.optimizers.SGD(
        learning_rate=lr_schedule,
        momentum=0.9
    )

    # Compile model with distributed optimizer
    model.compile(
        optimizer=optimizer,
        loss=keras.losses.SparseCategoricalCrossentropy(),
        metrics=[keras.metrics.SparseCategoricalAccuracy()]
    )
```

5.4 Input Pipeline Optimization

Listing 5.3: Optimized Data Pipeline

```
train_ds = (
    tf.data.Dataset.from_tensor_slices((x_train, y_train))
    .shuffle(20000, seed=SEED)
    .batch(BATCH_SIZE, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)
```

5.5 Results

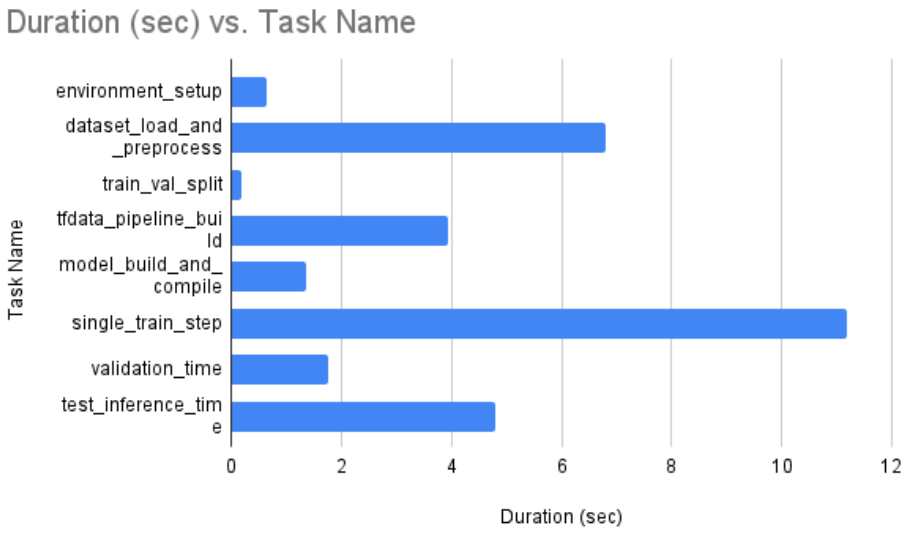

Figure 5.1: Execution Time Breakdown of the Parallel Training Pipeline

Chapter 6

Conclusion

This report has demonstrated the full spectrum of performance optimization in deep learning. We validated that low-level C API implementations provide superior inference latency for lightweight models, while high-level distributed strategies are indispensable for training complex architectures like ResNet-18.